DREW

Lecture 19: Assembly Programing

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

10/14/2024

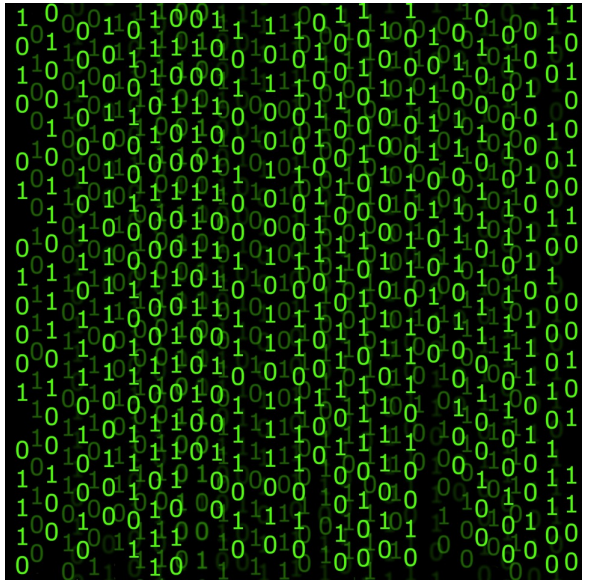
Important Dates

- **Next Lecture**:
  - In-Class Exercise with Attendance (Please bring your laptops.)

- **Midterm** on 10/30
- **Midterm Sample** available on 10/25; **Review** on 10/28

- **HW4** due on 10/25 by 11:59pm
- **Project1** due on 11/11 by 11:59pm

Instructions for Machine and for Human



- **Recall**: Machines only understand 0s and 1s.
- All the instructions are hence in 0s and 1s.

- That's not appealing to us, the programmers, those who tells the computer what to do! [electric genius/idiot?]
- All the programs need our directions (I know ML/DL/AI/GAN …); but deep down, each of them is an expert-system. [Use a CNN for NLP, likely not that effective.]

- So, we, programmer, use an equivalence form of the machine language, the *Assembly language*; simply instructions in *symbolic forms*.

Instructions for Machine and for Human

- **Ex.**: Instead of saying 0001110010000110, we say ADD R6, R2, R6.

- **Recall**: The bin/hex code is referred as "Raw".
- Turns out, if we input Assembly code, there's an *Assembler* program that turns our symbols to the corresponding 0s and 1s.

- Besides having more understandable representations, there are additional notions help us with programming in Assembly.

Assembly Example

Ex.:

```
; Program to multiply a number by the constant 6.
        .ORIG x3050          ; starts at x3050 location
        LD    R1, SIX        ; load #6 from labeled place containing #6
                             ; LC3 can interpret SIX as its PC+PCoffset9
        LD    R2, NUMBER     ; load the number from labeled place
        BRz   DONE           ; if the number is zero, we are done
; the inner loop
AGAIN   ADD   R3, R3, R2     ; adding R2 to the running sum
        ADD   R1, R1, #-1    ; decrement iter. var. by 1
        BRp   AGAIN          ; back to loop
DONE    HALT
NUMBER  .BLKW 1
SIX     .FILL x0006
        .END                 ; end of assembly program
```

DREW

Assembly Instruction Format

- Each line in the program is of the format:

  ; COMMENTS or

  Label OPCODE OPERANDS ; COMMENTS

- Labels (anything but reserved words) and comments (sort of) are optional.

- For each OPCODE, instead of giving 0s and 1s, simply give the symbol:
  ADD, AND, NOT, LEA, LD, ST, LDR, STI, BRp, etc.

- For each OPERAND: can have registers as simply type out R0-R7;
  can have numbers in dec as #16 or in hex as x5;
  can have labels by typing the label name LOOP.
  [OPERANDs are separated by commas.]

Assembly Instruction Format

New Commands: Pseudo-Ops

CODE	OPERAND	Functionality
.ORIG	address in hex	Specifying program starting addr.
.END	N/A	Indicating the end of program.
.FILL	# or x (16-bit wide)	Allocate a one-word wide numerical value here.
.BLKW	#n	Reserve a **BL**oc**K** of n **W**ords that can be allocated later.
.STRINGZ	n-char string	Allocation n+1 locations with n chars and a null terminator.

Pseudo-Ops Example

https://wchargin.com/lc3web/

Ex.: .BLKW

```
        .ORIG   x3200
WORD    .BLKW   #5
        LEA     R0, WORD  ; load R0 with EA of WORD
        LDR     R1, R0, #0  ; R1 ← Mem[Reg[R0]+0]
        LD      R2, WORD  ; R2 ← Mem[EA of WORD]
        .END
```

- **Note**: x3200-x3204 are *skipped*; i.e., not occupied by the following ops; and when executing, x3200-x3204 are *skipped*; not being treated as ops, even when they are filled, manually or via instructions.
- Now, let manually fill those fields and test:

  R0=x3200; R1=R2=Mem[x3200].

DREW

Pseudo-Ops Example

https://wchargin.com/lc3web/

Ex.: .STRINGZ

```
        .ORIG x3000
HELLO   .STRINGZ  "Hello, World!"    ; the string
        LEA       R1, HELLO  ; EA of 1st char
        ADD       R2, R2, #13  ; counter
LOOP    BRz       DONE
        LDR       R0, R1, #0   ; load 1st char
        TRAP      x21          ; display
        ADD       R1, R1, #1   ; update char addr
        ADD       R2, R2, #-1  ; update counter
        BRnzp     LOOP
DONE    HALT
        .END
```

BUT usually, data goes here.
Unless more than 255 spots.

DREW

Assembly Instruction Format

New Commands: TRAPs, System Routines

CODE	Same as	Functionality
HALT	TRAP x0025	Stopping system clock
IN	TRAP x0023	Print prompt to console and read from Keyboard, and store in R0.
OUT	TRAP x0021	Read from R0 and print to console.
GETC	TRAP x0020	IN w/o prompt.
PUTS	TRAP x0022	Print a null-terminated string to console. The EA of the string is in R0.

TRAP Example

https://wchargin.com/lc3web/

Ex.: .PUTS

```
.ORIG x3100
LEA         R0, HELLO ; EA of 1st char
PUTS
DONE        HALT
HELLO       .STRINGZ   "Hello, World!"     ; the string
.END
```

We are done!

DREW

TRAP Example

https://wchargin.com/lc3web/

Ex.: .GETC

```
        .ORIG    x3200
WORD    .BLKW    #5
        LEA      R1, WORD   ; load R0 with EA of WORD
        GETC
        OUT
        STR      R0, R1, #0    ; store char read to R1 location
        .END
```

How do Assembly Language work?

A Two-Pass Procedure

First Pass

- Scan program file line by line:
- Identify .ORIG.
- Find label, compute its *EA*, and generate a new row in the *Symbol Table*.
- Check Syntax errors: if duplicate/wrong label? if wrong OP/OPERANDs.
- Identify .END

Second Pass

- Convert to machine language with labels replaced by their EAs in PC+PCoffsets.

Symbol Table Example

Ex.:	Address	Label	Instruction
	… …		
	x3053	AGAIN	ADD R3, R3, R2
	x3054		ADD R1, R1, #-1
	x3055		BRp AGAIN
	x3056		HALT
	x3057	NUMBER	.BLWK 1
	x3058	SIX	.FILL x0006
	… …		

Symbol	Address
AGAIN	x3053
NUMBER	x3057
SIX	x3058

How do Assembly Language work?

An Assembly Program once successfully “Assembled”, generates two files:

- Object File (.obj): the machine code
- Symbol Table (.sym)
- [Need to add extensions from download.]

- To load program: Can upload both to the simulator at the same time.

Assembly Example

Ex.: Count the occurrences of a char in a file: (assume occurs at most 9 times.)

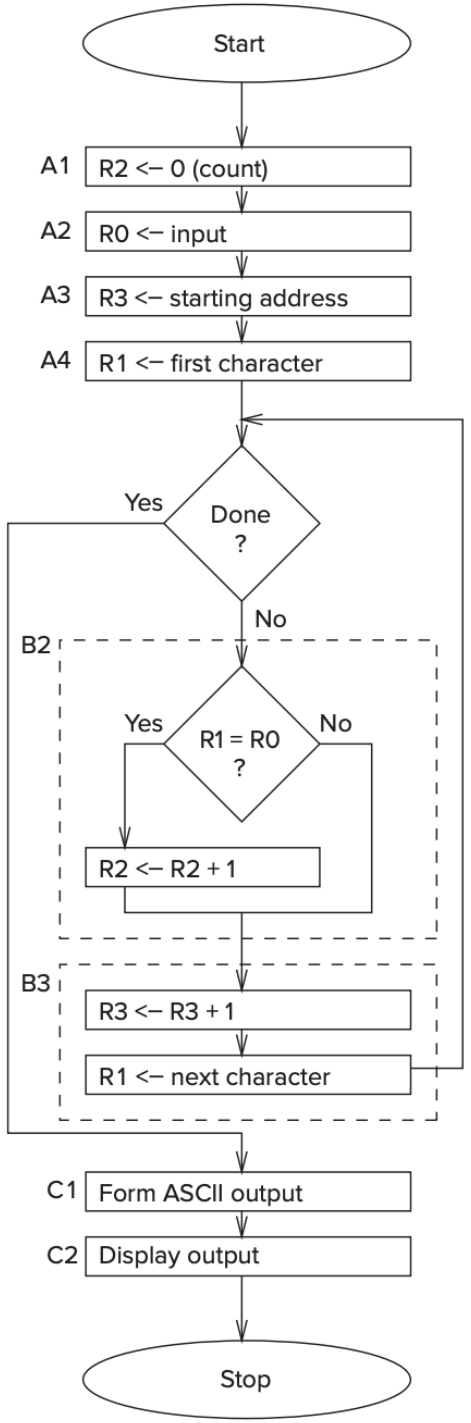
See program files on Moodle.

- File starts at x4000

```
.ORIG x4000
.STRINGZ "BARNARDO Who's there? FRANCISCO  Nay, answer me. Stand and unfold yourself."
.FILL x0004
.END
```

- End with an EOT (x0004) symbol.

- Then, input the char being counted.



Questions?

- **Next Lecture**:
  - In-Class Exercise with Attendance (Please bring your laptops.)

- **Midterm** on 10/30
- **Midterm Sample** available on 10/25; **Review** on 10/28

- **HW4** due on 10/25 by 11:59pm
- **Project1** due on 11/11 by 11:59pm